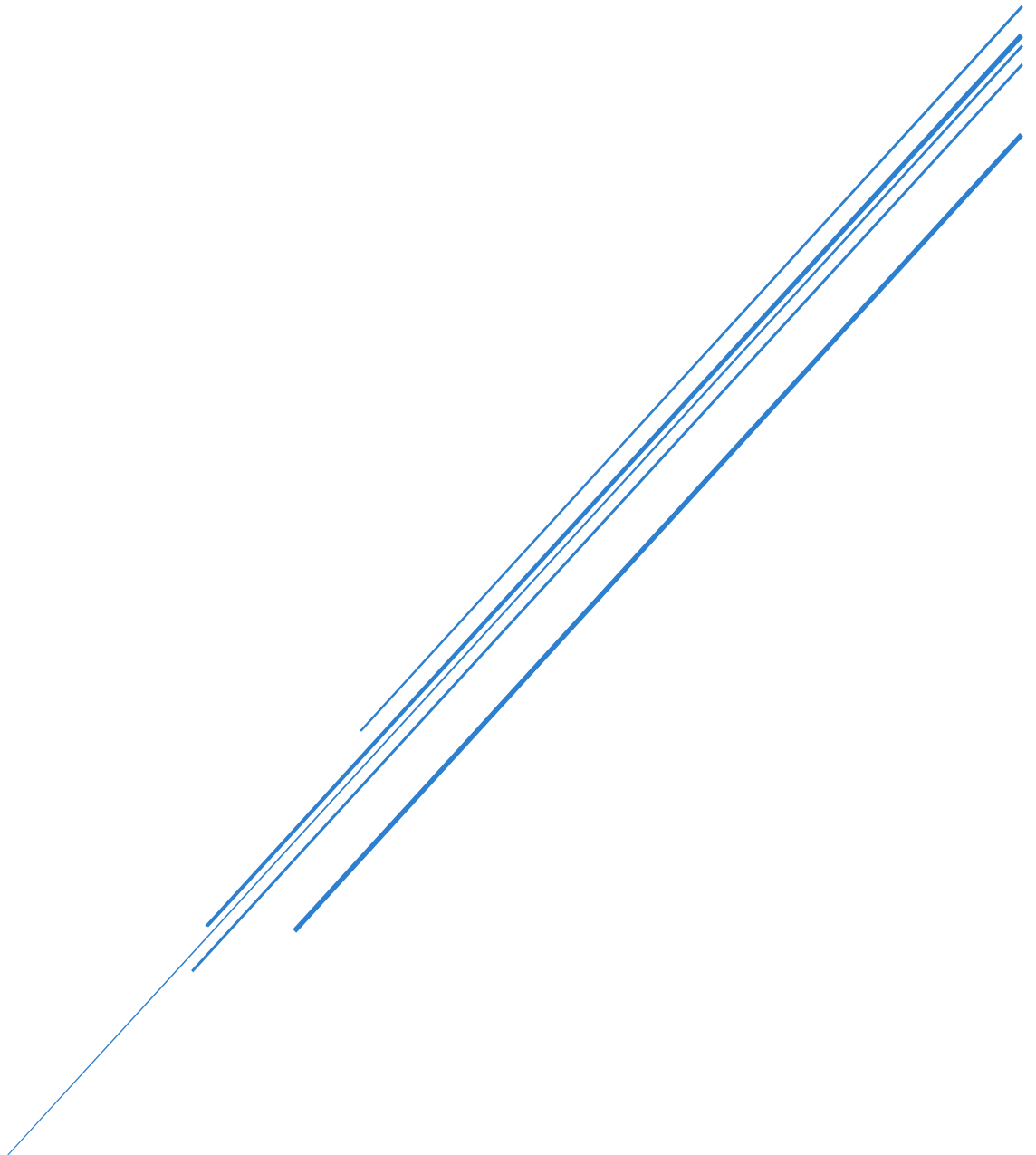


IMPLEMENTATION GUIDE: MATHEMATICAL MODELLING OF THE HUMAN BRAIN

Using MRI and Finite Element Simulation



Insight Research Services Associated
2025

Table of Contents

0. OVERVIEW	1
0.1. INTRODUCTION	1
0.2. METHODOLOGY	1
<i>Approach</i>	1
<i>Technical Challenges Encountered</i>	2
0.3. RESULTS.....	2
<i>Key Results</i>	2
<i>Experience with Software Implementation</i>	3
0.4. FUTURE WORK.....	3
<i>Computational Enhancements</i>	3
<i>Model Advancement</i>	4
<i>User Experience</i>	4
1. CHAPTER 1	5
1.1. OVERVIEW	5
2. CHAPTER 2	6
2.1. OVERVIEW	6
2.2. CODE CHANGES	6
<i>System Setup and Software Installation</i>	6
3. CHAPTER 3	10
3.1. OVERVIEW	10
3.2. CODE CHANGES	10
3.3. RESULTS.....	11
CHAPTER 4	15
4.1. OVERVIEW	15
4.2. CODE CHANGES	15
4.3. RESULTS.....	19
5. CHAPTER 5	24
5.1. OVERVIEW	24
5.2. CODE CHANGES	24
5.3. RESULTS.....	26
6. CHAPTER 6	30
6.1. OVERVIEW	30
6.2. CODE CHANGES	30
6.3. RESULTS.....	32
7. CHAPTER 7	42
7.1. OVERVIEW	42

0. Overview

0.1. Introduction

This document serves as an implementation guide for the computational methods presented in *Mathematical Modeling of the Human Brain: From Magnetic Resonance Images to Finite Element Simulation*. Following the book's section numbering, it documents practical challenges encountered during implementation and their solutions, serving as a companion reference for future interns.

The book outlines a comprehensive pipeline for constructing finite element models (FEM) of the brain from MRI data, covering data preprocessing, mesh generation, and the simulation of biophysical processes such as solute diffusion. This guide systematically validates these workflows, addressing real-world concerns such as software compatibility, data processing requirements, and numerical stability. Through practical testing and refinement, it provides verified computational procedures and optimization strategies.

This structured approach ensures reproducibility and provides a foundation for further advancements in computational brain modeling.

0.2. Methodology

This document adheres to the same section numbering as *Mathematical Modeling of the Human Brain: From Magnetic Resonance Images to Finite Element Simulation* to serve as a structured reference for the challenges encountered and solutions applied during the replication process.

Approach

The book was systematically read, and its methodologies were implemented in parallel to ensure full comprehension and reproducibility. Each section was followed closely, with code provided in the book executed, modified where necessary, and analysed to validate expected outcomes. At each stage, challenges were documented and addressed through targeted corrections and optimisations. The computational workflow involved the following steps:

1. **Environment Setup:** Installation and configuration of core software, including FreeSurfer, FEniCS, SVM-Tk, and supporting libraries.
2. **Data Processing:** Implementation of MRI processing pipeline for T1-weighted and DTI datasets, converting raw DICOM data into usable segmented formats.

3. **Mesh Generation:** Utilising FreeSurfer and SVM-Tk to generate finite element meshes from MRI segmentations.
4. **Simulation:** Implementing diffusion simulations using FEniCS, incorporating anisotropic diffusion tensors extracted from DTI data.
5. **Validation and Debugging:** Comparing simulation results with expected behaviour, troubleshooting discrepancies, and optimising computational performance.

Technical Challenges Encountered

During replication, significant technical challenges were encountered and resolved across several domains, including but not limited to:

- Software compatibility and dependency management
- MRI data processing and format conversion
- Mesh quality and generation
- FEM implementation and solver optimisation
- Resource management for computationally intensive operations

For ease of reference, this document maintains the book's section numbering system while documenting necessary modifications and solutions to technical challenges that were necessary to achieve successful replication of the book's computational pipeline.

0.3. Results

Key Results

The computational workflows implemented from the book generated several types of outputs:

Processing Outputs:

- Brain surface reconstructions from T1-weighted MRI data
- Tetrahedral volume meshes of varying resolutions
- DTI-derived diffusion tensor fields
- Region-specific brain parcellations

File Formats Generated:

- FreeSurfer outputs (.mgz, .stl)

- Mesh files (.mesh, .xdmf, .h5)
- Diffusion data (.nii.gz, .mgz)
- Simulation results (.xdmf)

Validation Steps:

- Surface segmentation quality checks
- Mesh integrity verification
- Numerical convergence testing
- Diffusion tensor field visualisation

Note: While many outputs serve as intermediate files for subsequent processing steps, detailed results and figures are presented in later sections of this document.

Experience with Software Implementation

Working with the book's software pipeline revealed both strengths and challenges:

Strengths:

- Comprehensive processing pipeline from raw MRI to simulation
- Well-documented workflow steps
- Integration of multiple open-source tools

Challenges:

- Memory-intensive processing requiring swap space allocation
- Software dependency conflicts requiring alternative installation methods
- Need for parameter adjustments to achieve stable simulations

0.4. Future Work

This section outlines potential areas for further research and development that could enhance the methodologies and applications presented in the book.

Computational Enhancements

- Investigate parallel computing and cloud-based solutions to improve processing efficiency for large datasets
- Explore alternative meshing strategies, including deep learning-based approaches, to improve anatomical accuracy

- Develop automated preprocessing pipelines to streamline workflows and enhance reproducibility

Model Advancement

- Validate simulations against experimental diffusion-weighted MRI data
- Integrate additional physiological factors (cerebrospinal fluid dynamics, brain elasticity)
- Compare performance across different finite element analysis platforms (COMSOL, ANSYS)

User Experience

- Create user-friendly interfaces for common preprocessing tasks
- Implement advanced visualisation tools for better result interpretation
- Develop comprehensive documentation and tutorials for new users

By addressing these areas, future work can build upon the foundation laid in this study, advancing the capabilities and applications of computational brain modeling.

1. Chapter 1

1.1. Overview

This chapter lays the groundwork by introducing the human brain's intricate structural and functional complexity. It emphasises the brain's multi-scale and multi-physical characteristics and motivates the use of computational models, especially continuum-based approaches, to explore phenomena such as solute diffusion and neurodegenerative disease mechanisms. By presenting a model problem based on anisotropic diffusion governed by partial differential equations, the chapter demonstrates how MRI data serves as a critical link between clinical imaging and computational simulations. Additionally, it introduces the primary open-source tools (e.g., FreeSurfer, SVM-Tk, FEniCS) and outlines the overall structure of the book, preparing readers for the hands-on workflows that follow.

2. Chapter 2

2.1. Overview

Chapter 2 builds essential foundational knowledge, starting with an overview of brain anatomy and the principles of MRI, including T1- and T2-weighted imaging for structural segmentation and diffusion tensor imaging (DTI) for assessing white matter anisotropy. The chapter explains how imaging data, stored in formats like DICOM, can be efficiently extracted using tools such as DicomBrowser. It also introduces the software ecosystem integral to the pipeline: FreeSurfer for processing MRI scans, SVM-Tk for generating high-quality meshes, and FEniCS for conducting finite element simulations. By providing installation guides and detailed workflows, this chapter equips readers with the technical know-how to transition from raw imaging data to simulation-ready models.

2.2. Code Changes

System Setup and Software Installation

To replicate the book's workflow, a suitable computational environment was set up. This involved downloading and installing the necessary software components on **WSL (Ubuntu 20.04)** for Linux-based execution.

```
wsl --install -d Ubuntu-20.04
```

Editing and Sourcing Bash Scripts:

- To edit the bash source script for future modifications, use:

```
$ nano ~/.bashrc
```

- After making changes, exit by pressing **Ctrl + X**, then **Y** to save.
- Apply the changes with:

```
$ source ~/.bashrc
```

Important Note: Before running any installation commands, ensure that the system's package list is up to date by running:

```
$ sudo apt-get update
```

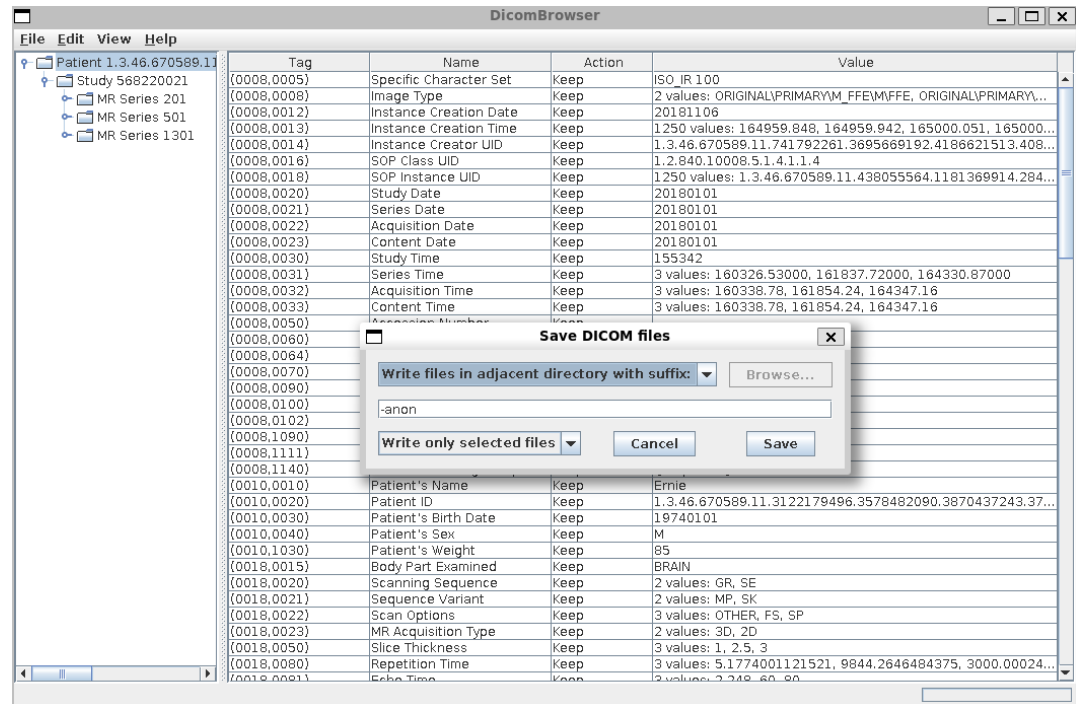
1. Python Installation:

- Installed **Python 3.11.7**, as certain dependencies required an updated Python version for compatibility.

2. DICOM Data Handling:

- Installed DICOM Browser in Linux Debian format to process MRI image storage.

The screenshot below shows the DicomBrowser interface used to inspect and configure DICOM tags for the selected MR series (dicom/ernie).



Here, you can see how individual DICOM fields are displayed and how the user can adjust the anonymisation settings before saving the files.

- A missing Java Virtual Machine (JVM) error was encountered and resolved using:

```
$ sudo apt install default-jre
$ sudo apt install default-jdk
```

3. FreeSurfer Installation:

- FreeSurfer was downloaded as a 64-bit tar archive for Ubuntu 20.04 and unpacked locally.
- The FreeSurfer environment was configured manually by adding the following lines at the end of the `.bashrc` file:

```
export  
FREESURFER_HOME=/mnt/c/Users/Intern.AzureAD/freesurfer  
  
export SUBJECTS_DIR=$FREESURFER_HOME/subjects  
  
source $FREESURFER_HOME/SetUpFreeSurfer.sh
```

- A **FreeSurfer license key** is required for the tools to function. A license key was obtained by registering at:
<https://surfer.nmr.mgh.harvard.edu/registration.html>
- After receiving the `license.txt` file via email, it was copied to the FreeSurfer installation directory, which matches the `FREESURFER_HOME` variable.

4. FSL Installation:

- Followed the official installation guide at:
<https://fsl.fmrib.ox.ac.uk/fsl/docs/#!/install/linux>
- Downloaded the `fslinstaller.py` script and navigated to the directory where it was saved:

```
$ cd Downloads  
$ python3 fslinstaller.py
```

- Configured the FSL environment:

```
FSLDIR=~/.fsl  
PATH=${FSLDIR}/share/fsl/bin:${PATH}  
export FSLDIR PATH  
. ${FSLDIR}/etc/fslconf/fsl.sh
```

5. ParaView Installation:

- ParaView was installed on Ubuntu Linux using the provided terminal command.
- Additionally, the Python 3 bindings for ParaView were installed for scripting support, though this was not mandatory:

```
$ sudo apt-get install python3-paraview
```

Installing SVMTK for Mesh Generation

The **Surface Volume Meshing Toolkit (SVMTK)** was required for generating finite element meshes. The book recommended compiling it from source, which required the following:

6. Installing SVMTK (Manual Compilation Method)

- Cloned the SVMTK repository recursively to include dependencies:

```
$ git clone --recursive https://github.com/SVMTK/SVMTK
```

- Installed required dependencies:

```
$ sudo apt-get install -y cmake libeigen3-dev libgmp3-dev  
libmpfr-dev libboost-all-dev
```

- Updated package manager for compatibility:

```
$ pip install -U setuptools
```

- Built and installed SVMTK:

```
$ sudo python3 setup.py install --verbose  
$ sudo python3 setup.py clean
```

Alternative SVMTK Installation Using Conda

While the book suggested compiling SVMTK manually, an **alternative approach** using Conda was found to be significantly easier. This method streamlined the process by handling dependencies automatically:

- Before proceeding, Miniconda for Linux was installed by following the official installation guide at: <https://docs.anaconda.com/miniconda/install/>

- Created and activated a Conda environment:

```
$ conda create --name myenv python=3.9  
$ conda activate myenv
```

- Installed SVMTK and additional dependencies:

```
$ conda install conda-forge::svmtk  
$ conda install nibabel scipy conda-forge::meshio conda-  
forge::fenics
```

This approach eliminated the need for manual dependency management and was recommended for future implementations due to its simplicity and reliability.

3. Chapter 3

3.1. Overview

In Chapter 3, the focus shifts to a detailed, step-by-step process for generating brain meshes from T1-weighted MRI data. The chapter explains how FreeSurfer is used to extract critical anatomical surfaces (like the pial boundary) and convert these surfaces into STL files for further processing. It then covers the application of SVM-Tk to remesh and smooth the extracted surfaces, addressing common issues such as surface defects. To illustrate the entire pipeline, a finite element simulation of gadobutrol diffusion is presented, demonstrating the definition of boundary conditions, the numerical solution of the diffusion equation with FEniCS, and the visualisation of results in ParaView. Practical aspects, including scripting for batch processing and handling numerical artifacts, are also discussed to prepare readers for real-world applications.

3.2. Code Changes

To correctly process and convert mesh files in Chapter 3, the following modifications were made to ensure compatibility with subsequent simulations and visualisation tools.

1. Correcting FreeSurfer Mesh Conversion Command:

- The book incorrectly states the command as:

```
$ mris_convert ./lh.pial pial.stl
```

- However, the correct command that preserves the proper naming convention is:

```
$ mris_convert ./lh.pial lh.pial.stl
```

2. Correcting Mesh Conversion with Meshio:

- The book references `meshio-convert`, which is outdated. The correct command structure in newer versions of the `meshio` package is:

```
$ meshio convert lh.mesh lh.xdmf
```

- This correction should be applied consistently throughout the book wherever `meshio-convert` is mentioned.

3. Copying DICOM Extraction Script to Dataset Directory:

- The script `extract_dicom.sh` is required for processing DICOM files. It needs to be copied into the appropriate dataset directory to ensure proper execution.

```
$ cp ../../../../mri2fem-dataset/dicom/ernie .
```

- This ensures that DICOM files are correctly extracted and placed in the expected directory for subsequent processing steps.

These code adjustments ensure that mesh conversion and data extraction steps align with the computational pipeline and avoid errors in later processing stages.

3.3. Results

1. T1 Series Extraction and Reconstruction

```
$ cd dicom/ernie/T13D  
$ recon-all -subjid ernie -i IM_0162 -all
```

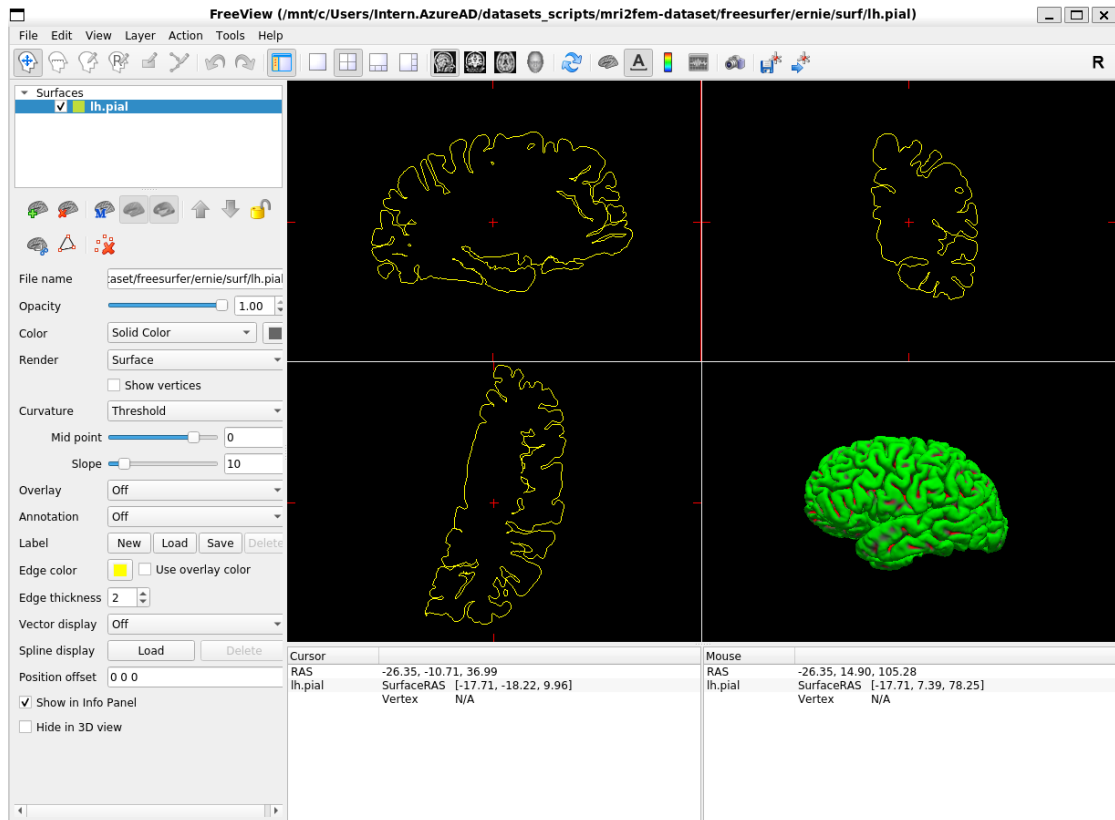
Sets the working directory to the T1 DICOM (T13D) folder and runs FreeSurfer's reconstruction on image IM_0162 using "ernie" as the subject ID. This produces volumetric segmentations, surface reconstructions (e.g. "lh.pial"), and statistical files that form the basis for later mesh generation.

2. Surface Inspection

```
$ cd freesurfer/ernie/surf  
$ freeview &
```

Changes the directory to the surfaces folder and launches Freeview to visually inspect reconstructed brain surfaces (e.g. the pial surface), confirming segmentation quality.

The following screenshot shows the Freeview output of the pial surface segmentation.



This screenshot displays a pial surface of the left brain hemisphere extracted from T1 images via FreeSurfer in Freeview, illustrating the reconstructed cortical folds from multiple viewing angles.

3. Surface Conversion to STL Format

```
$ mris_convert ./lh.pial lh.pial.stl
$ mris_convert ./lh.white lh.white.stl
```

Converts FreeSurfer's binary surface files ("lh.pial" and "lh.white") into STL files ("lh.pial.stl" and "lh.white.stl"), which are standard formats used for subsequent volume mesh generation.

4. Volume Mesh Generation:

```
$ python surface_to_mesh.py
```

Runs the SVM-Tk-based script to generate a finite element volume mesh from the processed STL surface, discretising the brain's interior for simulation.

5. Mesh Conversion for Simulation

```
$ meshio convert lh.mesh lh.xdmf
```

Converts the mesh from the native “.mesh” format (produced by SVM-Tk) into the “.xdmf” format, which is compatible with FEniCS. This conversion bridges the mesh-generation stage with the simulation stage by producing a file (“1h.xdmf”) that FEniCS can readily import.

6. Surface Preprocessing – Remeshing and Smoothing

```
$ python remesh_surface.py  
$ python smooth_surface.py
```

Remeshes the STL surface to improve element uniformity and smooths it to reduce noise, yielding processed surface files that are better suited for volume mesh generation.

7. Mesh Conversion for Simulation

```
$ meshio convert ernie.mesh ernie.xml  
$ meshio convert ernie.xml ernie.xdmf
```

Converts the mesh from SVM-Tk’s native “.mesh” format into XML and then into XDMF, producing “ernie.xdmf” which is compatible with FEniCS for finite element analysis.

8. Mesh Repair Using SVMTK Utilities

```
$ python svmtk_repair_utilities.py
```

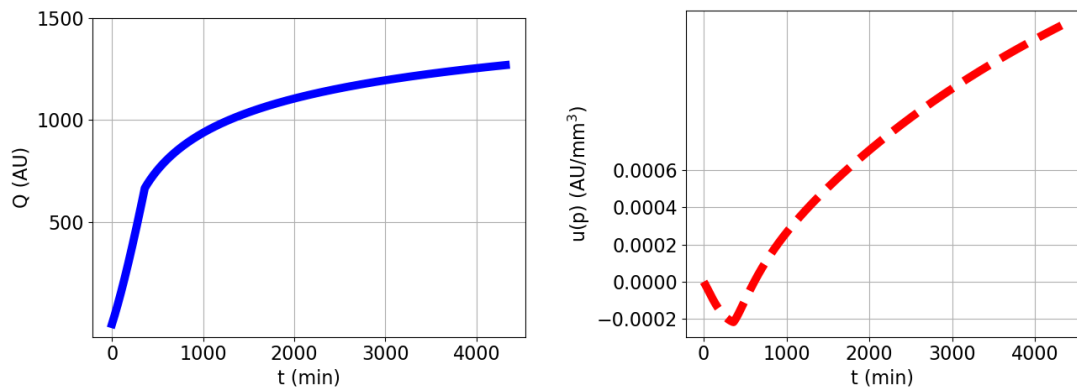
Executes a repair script that fixes common mesh defects (e.g. non-manifold edges or overlaps), resulting in a cleaner, more robust mesh for simulation.

9. Running the Diffusion Simulation

```
$ python diffusion.py
```

Executes the diffusion simulation script (using FEniCS) to model tracer diffusion in the brain; outputs simulation results such as concentration or displacement fields for further analysis.

The plots below show the total solute Q over time t (left) and the point concentration $u(p, t)$ for a given point p over time t (right), both in arbitrary units (AU).



The initial dip in the concentration (right) is a typical numerical artifact in diffusion simulations, caused by the early phase of the solver's initialisation.

10. DICOM Metadata Inspection – Interactive Viewing

```
$ cd dicom/ernie/DICOM
$ mri_probedicom --i IM_0162 --view
```

Navigates to the DICOM folder and displays metadata for IM_0162 using `mri_probedicom` in view mode, confirming imaging parameters and that the correct series is processed.

11. DICOM Tag Query

```
$ mri_probedicom --i IM_0162 --t 18 1030
```

Queries the DICOM file for tag 18 1030 (typically the protocol name), verifying that the image series T13D is correct.

12. Automated DICOM Series Extraction

```
$ cd dicom/ernie
$ ./extract_dicom.sh ./DICOM ./ "T1"
```

Changes to the DICOM root directory and runs a Bash script to automatically extract all DICOM files matching the “T1” identifier from the input folder, organising them into a designated output directory for streamlined processing.

Each command block in Chapter 3 incrementally transforms raw T1 MRI data into a high-quality, simulation-ready finite element model—bridging segmentation, surface processing, mesh generation, and simulation in a cohesive pipeline.

Chapter 4

4.1. Overview

Chapter 4 advances the modeling process by incorporating anatomical heterogeneities into the meshing techniques. It explains how to segment the brain into key regions: gray matter, white matter, and ventricles, using FreeSurfer, and describes methods for generating tagged meshes with SVM-Tk. The chapter covers advanced procedures such as merging hemispheres, repairing surface overlaps, and refining the mesh to balance accuracy with computational efficiency. A significant portion of the chapter is devoted to parcellation mapping, which involves transferring region-specific labels from the imaging data to the finite element meshes. Detailed instructions on data conversion (for example, converting from .mesh to .xdmf) and adaptive mesh refinement strategies ensure that the simulation models accurately capture the brain's complex anatomy.

4.2. Code Changes

To correctly process and refine mesh surfaces in Chapter 4, the following modifications were made to improve compatibility with later simulation steps.

1. Correcting FreeSurfer Mesh Conversion Command:

- The book incorrectly states the command as:

```
$ mris_convert ./lh.pial pial.stl
$ mris_convert ./lh.white white.stl
```

- However, the correct command that preserves the proper naming convention is:

```
$ mris_convert ./lh.pial lh.pial.stl
$ mris_convert ./lh.white lh.white.stl
```

2. Copying and Executing Remeshing and Smoothing Scripts:

- The book leaves this as an exercise for the reader, so this is the correct way to perform these steps
- Surface remeshing and smoothing are crucial for ensuring well-formed meshes. The required scripts from Chapter 3 need to be copied into the working directory:

```
$ cp ../chp3/remesh_surface.py .  
$ cp ../chp3/smooth_surface.py .
```

- Modify `remesh_surface.py` by adding:

```
remesh_surface("lh.white.stl", "lh.white.remesh.stl",  
1.0, 3)
```

- To add this modification to the `remesh_surface.py` script, use:

```
$ nano remesh_surface.py
```

- This approach should be used for future script edits:

```
$ nano your_script_name.py
```

- Execute the modified remeshing script:

```
$ python remesh_surface.py
```

- Similarly, modify `smooth_surface.py` by adding:

```
smoothen_surface("lh.white.remesh.stl",  
"lh.white.smooth.stl",  
n=10, eps=1.0)
```

- Execute the modified smoothing script:

```
$ python smooth_surface.py
```

- Rename files for consistency and easier reference:

```
$ mv lh.pial.smooth.stl lh.pial.stl  
$ mv lh.white.smooth.stl lh.white.stl
```

3. Fixing Ventricle Extraction Script:

- The script `extract-ventricles.sh` originally contains:

```
output="ventricles.stl"
```

- It should be modified to:

```
output="ernie-ventricles.stl"
```

- This ensures consistency in naming conventions across the pipeline.

4. Ensuring Mesh Conversion Before Running `map_parcellation.py`

- Before running `map_parcellation.py`, the mesh must first be converted to **XDMF format** to avoid errors:

```
$ meshio convert ernie-brain-32.mesh ernie-brain-32.xml
$ meshio convert ernie-brain-32.xml ernie-brain-32.xdmf
```

- If the mesh is not converted, attempting to run `map_parcellation.py` will result in:

```
Error: Unable to open HDF5 file.
Reason: File "ernie-brain-32.h5" does not exist.
```

- This step ensures that all necessary files exist before executing `map_parcellation.py`.
- Alternatively, you can copy `SVMTk-tet-mesh-to-FEniCS-XDMF.py` from `chp3` and paste the file into the `chp4` folder, then run:

```
$ cp ../chp3/SVMTk-tet-mesh-to-FEniCS-XDMF.py .
$ python SVMTk-tet-mesh-to-FEniCS-XDMF.py ernie-brain-32.mesh
```

5. Changes to `convert_to_dolfin_mesh.py` to Ensure Unique and Informative Output Filenames:

- The modified version now dynamically names output files based on the input mesh filename to prevent overwriting and improve organisation. Additionally, function interfaces and file operations have been updated to incorporate this new naming scheme.
- Extracting the base name from the input mesh file:

```
base_name = meshfile.rsplit('.', 1)[0]
```

- Adjusting file writing operations to include `base_name` in filenames:

```
meshio.write("%s/mesh_%s.xdmf" % (xdmfdir, base_name),
             xdmf)
meshio.write("%s/subdomains_%s.xdmf" % (xdmfdir,
                                         base_name), xdmf)
```

```
meshio.write("%s/boundaries_%s.xdmf" % (xdmfdir,
base_name), xdmf)
```

- Updating function signatures to accept `base_name`:

```
def write_xdmf_to_h5(xdmfdir, hdf5file, base_name):
```

- Updating file reading operations to use the new filenames:

```
with df.XDMFFile("%s/mesh_%s.xdmf" % (xdmfdir,
base_name)) as infile:
    infile.read(mesh)

with df.XDMFFile("%s/subdomains_%s.xdmf" % (xdmfdir,
base_name)) as infile:
    infile.read(subdomains, "subdomains")

with df.XDMFFile("%s/boundaries_%s.xdmf" % (xdmfdir,
base_name)) as infile:
    infile.read(boundaries, "boundaries")
```

- Modifying the main execution block to extract and pass `base_name`:

```
base_name = Z.meshfile.rsplit('.', 1)[0]
write_mesh_to_xdmf(Z.meshfile, Z.xdmfdir)
write_xdmf_to_h5(Z.xdmfdir, Z.hdf5file, base_name)
```

- These changes ensure that multiple meshes can be processed without file conflicts and provide better traceability of mesh conversions.

6. Fixing `convert_to_dolfin_mesh.py` Execution

- The book states:

```
$ python convert_to_dolfin_mesh.py --meshfile ernie-gw.mesh
--hdf5file ernie-gw.h5
```

- Running this results in a **FileNotFoundError** due to the missing `tmp/` directory. The solution is to either create the folder beforehand or explicitly specify the working directory:

```
$ python convert_to_dolfin_mesh.py --meshfile ernie-gw.mesh
--hdf5file ernie-gw.h5 --xdmfdir .
```

- Similarly, for the **full brain mesh**, the book incorrectly suggests:

```
$ python3 convert_to_dolfin_mesh.py --meshfile ernie-brain-32.mesh --hdf5file ernie-brain-32.h5
```

- This should be updated to:

```
$ python convert_to_dolfin_mesh.py --meshfile ernie-brain-32.mesh --hdf5file ernie-brain-32.h5 --xdmfdir .
```

These refinements ensure that surface processing and mesh conversions are performed correctly, leading to improved compatibility with finite element simulations.

4.3. Results

1. Surface Conversion to STL Format

```
$ mris_convert ./lh.pial lh.pial.stl  
$ mris_convert ./lh.white lh.white.stl
```

Converts FreeSurfer's binary surface files for the left hemisphere's pial and white matter interfaces into STL files ("lh.pial.stl" and "lh.white.stl"), which serve as inputs for subsequent volume mesh generation.

2. Creating a Gray-White Matter Mesh

```
$ python two-domain-tagged.py
```

Runs a Python script (using SVM-Tk) that reads the STL surfaces and generates a tagged volume mesh (e.g. "ernie-gw.mesh") distinguishing gray from white matter. This mesh forms the basis for simulating heterogeneous tissue properties.

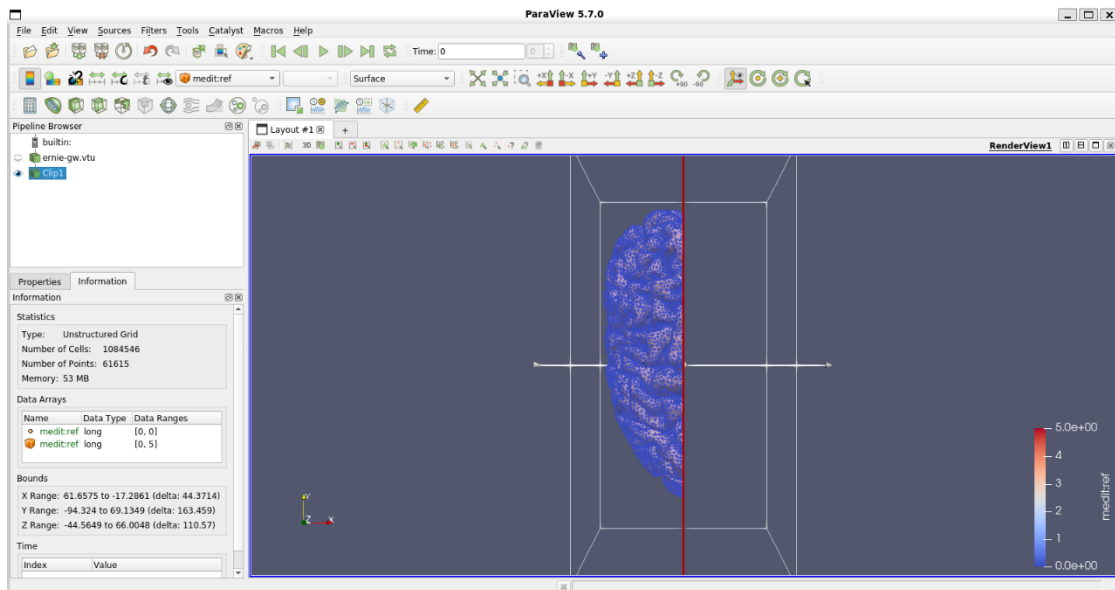
3. Mesh Conversion for Visualisation

```
$ meshio convert ernie-gw.mesh ernie-gw.vtu
```

Converts the heterogeneous volume mesh from its native ".mesh" format to VTU ("ernie-gw.vtu"), making it compatible with visualisation tools (e.g. ParaView) to inspect the tagged regions.

4. Visualising the Gray-White Mesh Tags in ParaView

The screenshot below shows the ernie-gw.vtu mesh in ParaView with a clipping filter applied to reveal the subdomain tags for gray (value = 1) and white matter (value = 2).



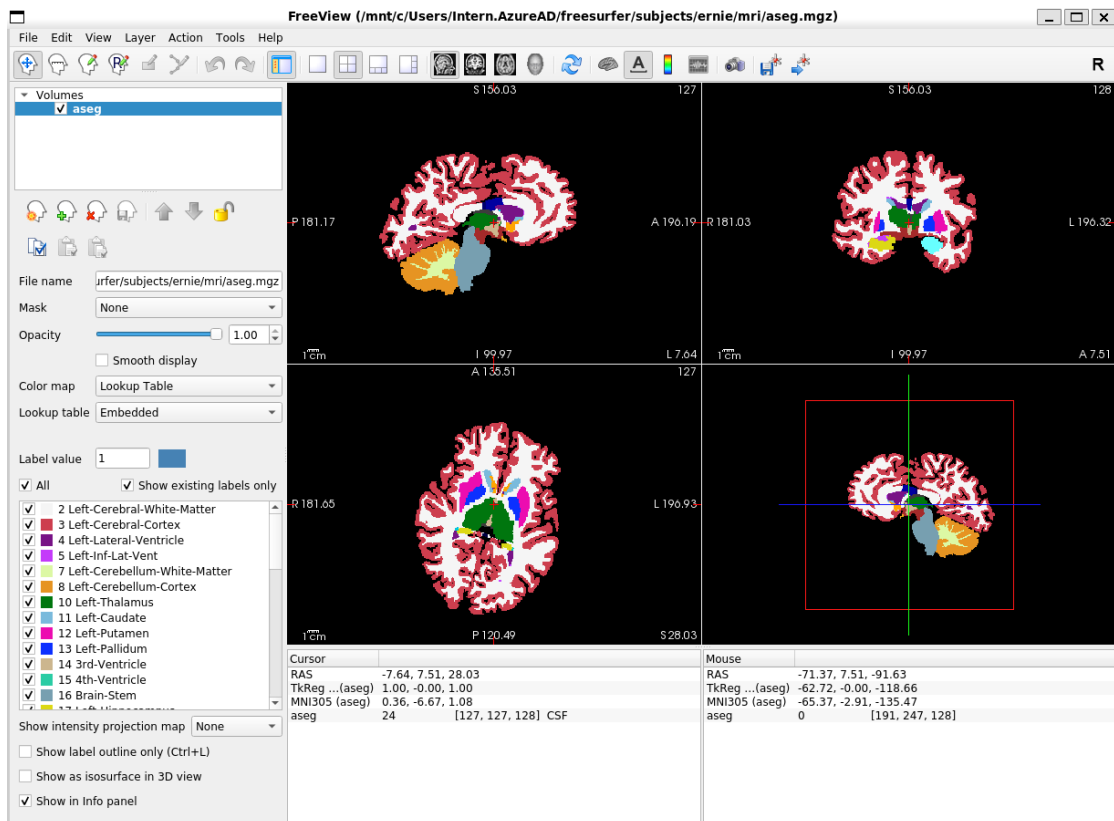
This view highlights how the mesh cells are distinguished by colour based on their tagged region, confirming the correct mapping of subdomains for simulation.

5. Segmentation Data Inspection

```
$ cd freesurfer/ernie/mri
$ freeview --colormap lut --v aseg.mgz
```

Changes the directory to the MRI folder and opens “aseg.mgz” in Freeview with a lookup table colormap to visualise segmentation labels, confirming the quality of the automatic segmentation.

The screenshot below shows “aseg.mgz” loaded in Freeview with a lookup table colormap, highlighting various segmented brain regions.



The left-hand panel lists the segmentation tags (e.g., tag 17 for the left hippocampus, tag 15 for the fourth ventricle), and hovering over a voxel displays its region tag and name in the bottom right corner.

6. Ventricular Volume Extraction from Segmentation

```
$ mri_binarize --i aseg.mgz --ventricles --match 15 --o v.mgz
```

Uses `mri_binarize` to extract the ventricular regions from “aseg.mgz” by matching tag 15; outputs “v.mgz,” a binary volume file representing the ventricles for subsequent meshing.

7. Extracting the Ventricular Surface

```
$ ./extract-ventricles.sh
```

Executes a Bash script that uses FreeSurfer utilities (e.g. `mri_binarize`) to extract, smooth, and output an STL file representing the ventricular surface. This file is used later to distinguish the ventricles from other brain tissues.

8. Creating a Three-Domain Mesh (Excluding Ventricles):

```
$ python three-domain-tagged.py
```

Runs a Python script that combines the pial, white, and ventricular STL surfaces to generate a volume mesh with three tagged regions, then removes the ventricular region. The resulting mesh (e.g. “lh.no-ventricles.mesh”) represents brain tissue excluding the ventricles.

9. Combining Hemispheres into a Full-Brain Mesh

```
$ python fullbrain-five-domain.py
```

Executes a Python script that loads STL files for both hemispheres (pial and white surfaces, plus the ventricles), performs union and repair operations to resolve overlaps, and produces a unified full-brain mesh (e.g. “ernie-brain-32.mesh”) ready for further processing.

10. Mapping Parcellation Data onto the Mesh

```
$ python map_parcellation.py
```

Runs a Python script that reads voxel-based parcellation data (e.g. “wmparc.mgz”) and maps anatomical labels onto the full-brain mesh (e.g. “ernie-brain-32.xdmf”), assigning region-specific tags to mesh cells for heterogeneous simulation.

11. Mesh Conversion to Dolfin Format (Gray-White Mesh)

```
$ cd mri2fem/chp4  
$ python convert_to_dolfin_mesh.py --meshfile ernie-gw.mesh --  
hdf5file ernie-gw.h5 --xdmdir .
```

Changes to the Chapter 4 scripts directory and converts the gray-white mesh (“ernie-gw.mesh”) into Dolfin-compatible formats, producing an HDF5 file (“ernie-gw.h5”) and associated XDMF files for finite element analysis with FEniCS.

12. Mesh Conversion and Parcellation Mapping for Full-Brain Mesh

```
$ python convert_to_dolfin_mesh.py --meshfile ernie-brain-  
32.mesh --hdf5file ernie-brain-32.h5 --xdmdir .  
$ python add_parcellations.py --in_hdf5 ernie-brain-32.h5 --  
in_parc wmparc.mgz --out_hdf5 results/ernie-brain-subdomains-  
tags.h5 --add 17 1028 1035 3028 3035
```

First, converts the full-brain mesh (“ernie-brain-32.mesh”) into Dolfin-compatible formats (producing “ernie-brain-32.h5” and corresponding XDMF files). Then, maps parcellation labels from “wmparc.mgz” onto the full-brain

mesh, outputting an updated HDF5 file (“results/ernie-brain-subdomains-tags.h5”) with enhanced subdomain tags for heterogeneous simulation.

13. Global Mesh Refinement of the Parcellated Mesh

```
$ python refine_mesh_tags.py --in_hdf5 ernie-brain-32.h5 --  
out_hdf5 ernie-brain-32-refined.h5
```

Executes a Python script that globally refines the mesh (and its associated subdomain data), producing a refined mesh file (“ernie-brain-32-refined.h5”) with increased overall resolution for improved simulation accuracy.

14. Local Mesh Refinement for Specific Regions

```
$ python refine_mesh_tags.py --in_hdf5 ernie-brain-32.h5 --  
out_hdf5 ernie-brain-32-refine-tags.h5 --refine_tag 17 1028  
1035 3028 3035
```

Runs the refinement script with parameters to locally refine only the mesh cells tagged with specified region codes, resulting in a locally refined mesh (“ernie-brain-32-refine-tags.h5”) that enhances resolution in areas of interest.

Each command block in Chapter 4 incrementally transforms segmented MRI surfaces into a detailed, heterogeneous finite element model—incorporating tissue interfaces, ventricular extraction, parcellation mapping, and adaptive mesh refinement—to produce a simulation-ready brain mesh for biomechanical analysis.

5. Chapter 5

5.1. Overview

Chapter 5 integrates diffusion tensor imaging (DTI) data into the computational framework, providing a comprehensive guide on leveraging DTI for enhanced modeling accuracy. It outlines the preprocessing steps in FreeSurfer necessary to extract critical tensor metrics such as eigenvalues, fractional anisotropy, and mean diffusivity. The chapter discusses techniques for co-registering DTI data with T1-weighted images and addresses common challenges like correcting non-physical tensor values and handling missing data. Detailed instructions are provided on mapping DTI-derived anisotropic diffusion tensors onto finite element meshes using Python tools like Nibabel. By tackling issues such as coordinate system alignment and voxel-to-mesh interpolation, this chapter ensures that the simulations can accurately reflect the directional properties of brain tissue.

5.2. Code Changes

To correctly process DTI data in Chapter 5, the following modifications were made to improve compatibility with later simulations and ensure correct data handling.

1. Setting Up the FreeSurfer Subjects Directory:

- FreeSurfer requires the `SUBJECTS_DIR` variable to be correctly set before processing DTI data.

```
$ export  
SUBJECTS_DIR=/mnt/c/Users/Intern.AzureAD/datasets_scripts/mr  
i2fem-dataset/freesurfer/
```

- The book incorrectly states the `dt_recon` command as:

```
$ dt_recon --i dti.mgz --b dti.bvals dti.voxel_space.bvecs -  
-s ernie --o $SUBJECTS_DIR/ernie/dti
```

- However, this results in an error: **cannot find dti.voxel_space.bvecs**. The correct command is:

```
$ dt_recon --i dti.mgz --b dti.bvals dti.scanner_space.bvecs  
--s ernie --o $SUBJECTS_DIR/ernie/dti
```

- This ensures the correct reconstruction of diffusion tensor data from the MRI scans.

2. Copying the Required Mask File:

- The `mask.mgz` file, generated in FreeSurfer's processing pipeline, must be copied to the Chapter 5 directory before running subsequent scripts.

```
$ cp ../../../../mri2fem-  
dataset/freesurfer/ernie/mri/wmparc.mgz .
```

- This ensures that the required mask is available for use in preprocessing the DTI data

3. Cleaning the DTI Data:

- The book incorrectly suggests the following command:

```
$ python3 clean_dti_data.py --dti tensor.nii.gz --mask  
mask.mgz --out tensor-clean.nii
```

- However, the output format should be in `.mgz` to align with FreeSurfer conventions. The correct command is:

```
$ python clean_dti_data.py --dti tensor.nii.gz --mask  
mask.mgz --out clean-tensor.mgz
```

4. Checking the Processed DTI Data and Mapping to the Mesh:

- First, verify the cleaned DTI data with:

```
$ python check_dti.py --dti clean-tensor.mgz --mask mask.mgz
```

- Copy the necessary finite element mesh file from Chapter 4:

```
$ cp ../chp4/ernie-brain-32.h5 .
```

- Convert the cleaned DTI data to a format compatible with the mesh:

```
$ python dti_data_to_mesh.py --mesh ernie-brain-32.h5 --dti  
clean-tensor.mgz --out ernie-brain-dti.h5
```

5. Verifying DTI Orientation Information:

- The book incorrectly suggests using:

```
$ mri_info tensor.nii.mgz --orientation
```

- However, this results in an error: **could not open file tensor.nii.mgz**. The correct command is:

```
$ mri_info tensor.nii.gz --orientation
```

These refinements ensure that DTI data is properly cleaned, processed, and integrated into the finite element simulation pipeline for further analysis.

5.3. Results

1. DTI Volume Conversion

```
$ cd dicom/ernie/DTI  
$ mri_convert IM_1496 dti.mgz
```

Sets the working directory to the DTI folder and converts the selected DICOM file (IM_1496) into a FreeSurfer volume file (“dti.mgz”), which serves as the input for DTI reconstruction.

2. DTI Reconstruction with FreeSurfer

```
$ export  
SUBJECTS_DIR=/mnt/c/Users/Intern.AzureAD/datasets_scripts/mri2  
fem-dataset/freesurfer/  
$ dt_recon --i dti.mgz --b dti.bvals dti.scanner_space.bvecs -  
-s ernie --o $SUBJECTS_DIR/ernie/dti
```

Reconstructs the full diffusion tensor field from “dti.mgz” using associated b-values and b-vectors; outputs files such as “tensor.nii.gz”, “eigvals.nii.gz”, “adc.nii.gz”, and “fa.nii.gz” that encapsulate the diffusion information for further processing.

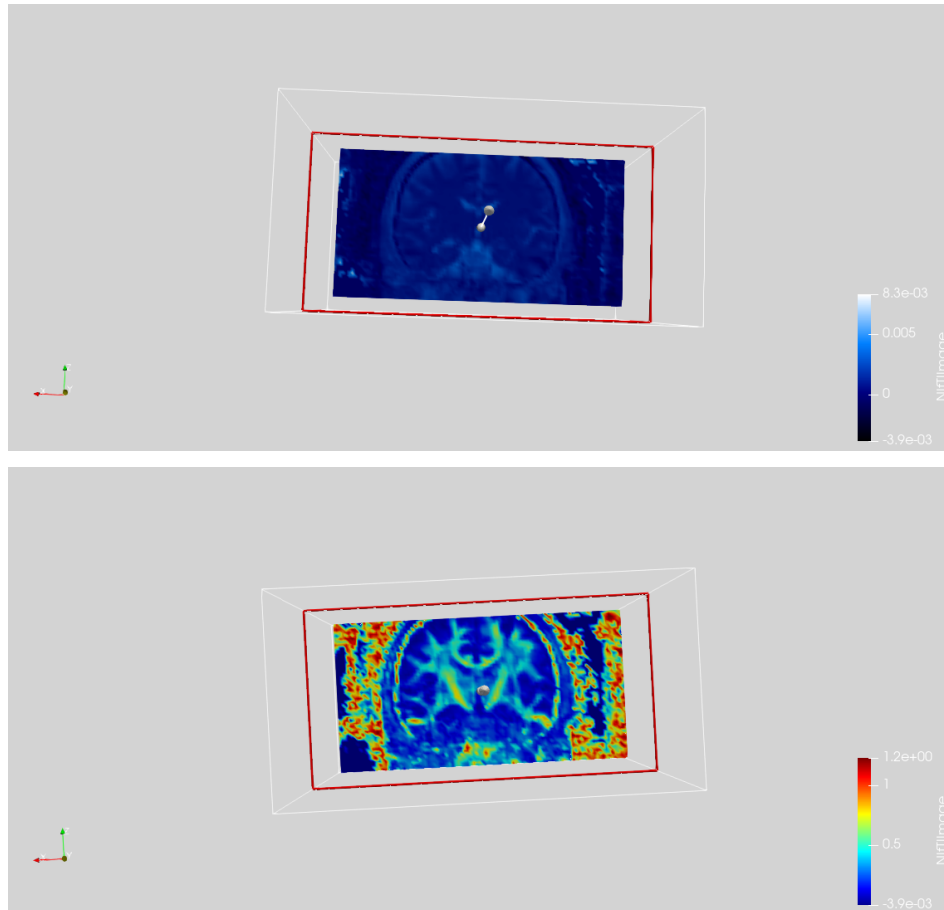
3. Converting ADC and FA Files

```
$ mri_convert adc.nii.gz adc.nii  
$ mri_convert fa.nii.gz fa.nii
```

Converts the compressed ADC and FA images from NIfTI (.nii.gz) to uncompressed (.nii) format, facilitating easier visualisation and processing of these diffusion metric.

4. Visualising NIfTI Data in ParaView

The images below show how adc.nii (top) and fa.nii (bottom) appear in ParaView after enabling the NIfTI reader plugin, volume rendering the data, and applying a slice filter.



These views confirm that the diffusion-weighted volumes can be conveniently loaded, sliced, and colour-mapped in ParaView to replicate the style of the figure in the book.

5. Creating a Gray Matter Mask from Parcellation

```
$ mri_binarize --i wmparc.mgz --gm --dilate 2 --o mask.mgz
```

Extracts the gray matter (GM) from the parcellation file “wmparc.mgz” (with dilation by 2 voxels) and outputs “mask.mgz”. This mask is later used to check and clean the DTI data.

6. Initial DTI Data Check Using the Mask

```
$ cd mri2fem/chp5
$ python check_dti.py --dti tensor.nii.gz --mask mask.mgz
```

Changes to the Chapter 5 scripts directory and runs a script to validate the raw tensor data (“tensor.nii.gz”) using the GM mask (“mask.mgz”). It outputs diagnostic information to assess data quality.

7. Cleaning the DTI Data

```
$ python clean_dti_data.py --dti tensor.nii.gz --mask mask.mgz  
--out clean-tensor.mgz
```

Executes a Python script that cleans the raw DTI tensor data using the mask, outputting “clean-tensor.mgz” for improved data quality in subsequent simulations.

8. Re-checking Cleaned DTI Data & Integrating with the Mesh

```
$ python check_dti.py --dti clean-tensor.mgz --mask mask.mgz  
$ cp ../chp4/ernie-brain-32.h5 .  
$ python dti_data_to_mesh.py --mesh ernie-brain-32.h5 --dti  
clean-tensor.mgz --out ernie-brain-dti.h5
```

First, re-runs the check_dti.py script to verify the cleaned tensor data. Then, copies the full-brain mesh (“ernie-brain-32.h5”) from Chapter 4 into the current directory and runs a script that integrates the cleaned DTI data with the mesh, producing “ernie-brain-dti.h5”—a simulation-ready mesh incorporating anisotropic diffusion properties.

9. Checking Orientation of Original MRI Data

```
$ cd $SUBJECTS_DIR/ernie/mri  
$ mri_info orig.mgz --orientation
```

Navigates to the subject’s MRI folder and displays the orientation of the original volume (“orig.mgz”), ensuring the MRI data’s alignment for co-registration with DTI data.

10. Checking Orientation of DTI Tensor Data

```
$ cd $SUBJECTS_DIR/ernie/dti  
$ mri_info tensor.nii.gz --orientation
```

Changes to the DTI directory and displays the orientation of “tensor.nii.gz”, which is essential for verifying proper alignment relative to the original MRI data.

11. Checking Voxel Sizes of DTI Data

```
$ cd $SUBJECTS_DIR/ernie/dti  
$ mri info tensor.nii.gz | grep voxel\ sizes
```

Queries the tensor file for its voxel size information, ensuring that spatial resolution is consistent for registration and simulation purposes.

12. Checking Voxel Sizes of Original MRI Data

```
$ cd $SUBJECTS_DIR/ernie/mri  
$ mri_info orig.mgz | grep voxel\ sizes
```

Queries the original MRI file for voxel size details, facilitating comparison with the DTI data and confirming proper spatial calibration.

Each command block in Chapter 5 incrementally transforms raw DTI DICOM data into processed diffusion tensor images and derived scalar maps (ADC and FA), and then integrates these data with a pre-generated finite element brain mesh. This pipeline bridges image conversion, tensor reconstruction, data cleaning, quality checks, and mesh integration—preparing the data for simulation of anisotropic diffusion in brain tissue.

6. Chapter 6

6.1. Overview

Focusing on the simulation aspect, Chapter 6 demonstrates how to perform anisotropic diffusion simulations within 3D brain models that integrate both MRI-derived meshes and DTI data. The chapter contrasts simpler one-dimensional analytical solutions with more sophisticated three-dimensional numerical approaches, addressing important issues like numerical stability, mass lumping, and adaptive mesh refinement. Through case studies that track the distribution of gadobutrol, particularly in complex regions like the hippocampus, the chapter illustrates how tissue heterogeneity affects diffusion patterns. It also includes mesh convergence analyses, highlighting the trade-offs between resolution and computational cost, and emphasises the need for validating simulation results against experimental data to ensure accuracy and reliability in modeling complex diffusion processes.

6.2. Code Changes

To ensure proper execution of diffusion simulations in Chapter 6, the following modifications were made to improve compatibility, performance, and memory handling.

1. Replacing SciPy with NumPy:

- If `analytical_1D.py` is run as is, the following error occurs:

```
ImportError: cannot import name 'sqrt' from 'scipy'
```

- To fix this, the first two `scipy` imports must be changed to `numpy`:

```
import numpy
from numpy import sqrt
```

- Additionally, all instances of `scipy.arange` must be replaced with:

```
numpy.arange()
```

- Similarly, if `diffusion_1D.py` is run as is, the following error occurs:

```
AttributeError: Module 'scipy' has no attribute 'arange'
```

- To resolve this, change the `scipy` import to `numpy`:

```
import numpy
```

- Then, update all occurrences of `scipy.arange` to:

```
numpy.arange()
```

- These changes prevent script failures.

2. Executing Diffusion Simulations with Various Parameters:

- The book provides several configurations for running `diffusion_1D.py`. The following commands execute diffusion simulations with increasing spatial resolution (`Ns`) and time durations (`final_time`):

```
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --final_time 1800 --lumped not --check_point 0.1
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --final_time 1800 --lumped lumped --check_point 0.1
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --final_time 32400 --lumped not --check_point 0.1
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --final_time 32400 --lumped lumped --check_point 0.1
```

3. Updating `all.sh` to Use the Correct Parameter Name:

- If `all.sh` is run as is, the following error occurs:

```
error: unrecognized arguments: --label
```

- To resolve this, change all instances of `--label` in `all.sh` to:

```
--annotation
```

- This ensures compatibility with `chp6-diffusion-mritracer.py` and `chp6-diffusion-mritracer-ref.py`.

4. Optimising Memory Usage by Adding Swap Space:

- Running `all.sh` in Chapter 6 requires more than 16GB of RAM, and without sufficient memory, execution is **killed** due to memory exhaustion.
- To prevent crashes, additional swap space can be allocated as follows:

```
$ sudo falloccate -l 20G /swapfile
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
$ sudo swapon /swapfile
```

- This enables the system to use disk space as additional memory, preventing failures due to RAM limitations.

These refinements ensure that diffusion simulations run efficiently and avoid errors related to memory limitations, incorrect imports, or outdated parameter names.

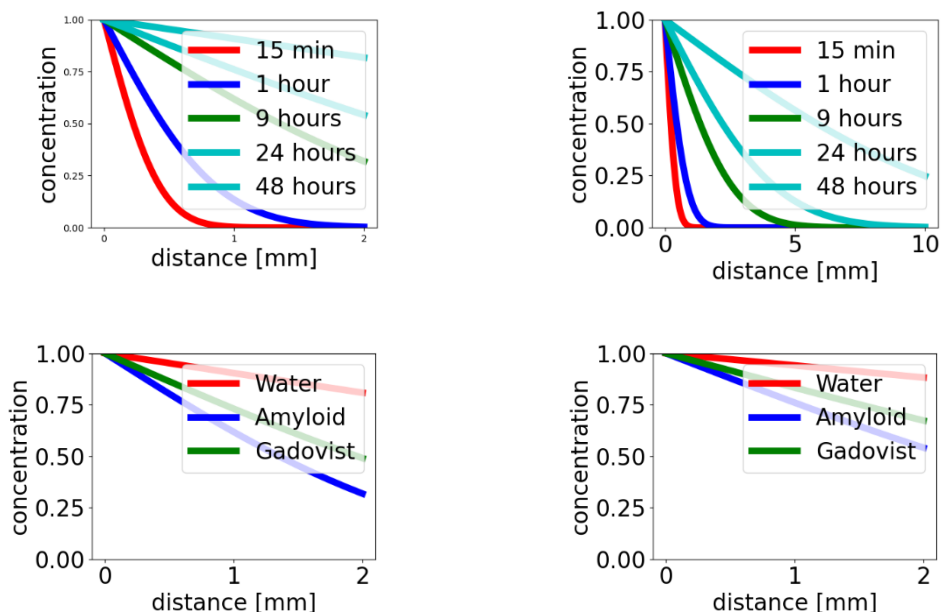
6.3. Results

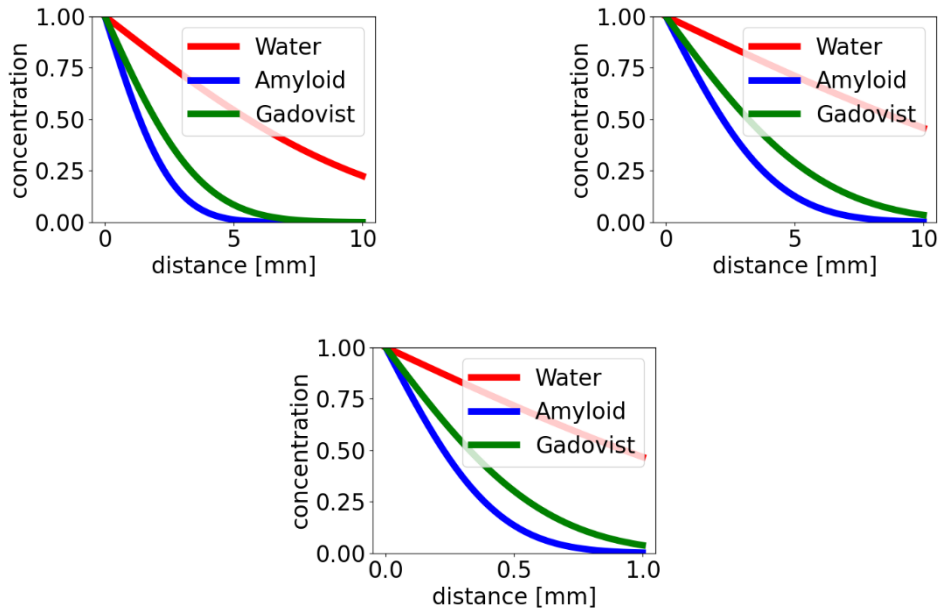
Preliminary: 1D Analytical Simulations

```
$ python analytical 1D.py
```

Runs an analytical one-dimensional diffusion model to compute a reference solution. This baseline output is used to validate and compare the numerical diffusion simulations.

The following seven plots (generated by `python analytical 1D.py`) show the 1D analytical solutions of equation (6.1) in the book for water, amyloid-beta, and gadobutrol at various time points (up to 24 hours) and distances (up to 10 mm). They illustrate how each molecule's concentration diminishes with increasing distance, highlighting the slower diffusion rates of the heavier molecules.





These results confirm that diffusion is a relatively slow process: within a few millimetres of the boundary, concentrations change significantly over several hours, while at greater distances (e.g., 1 cm), heavier molecules such as amyloid-beta and gadobutrol still have near-zero concentration.

Preliminary: 1D Diffusion Simulations

```
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --
final_time 1800 --lumped not --check_point 0.1

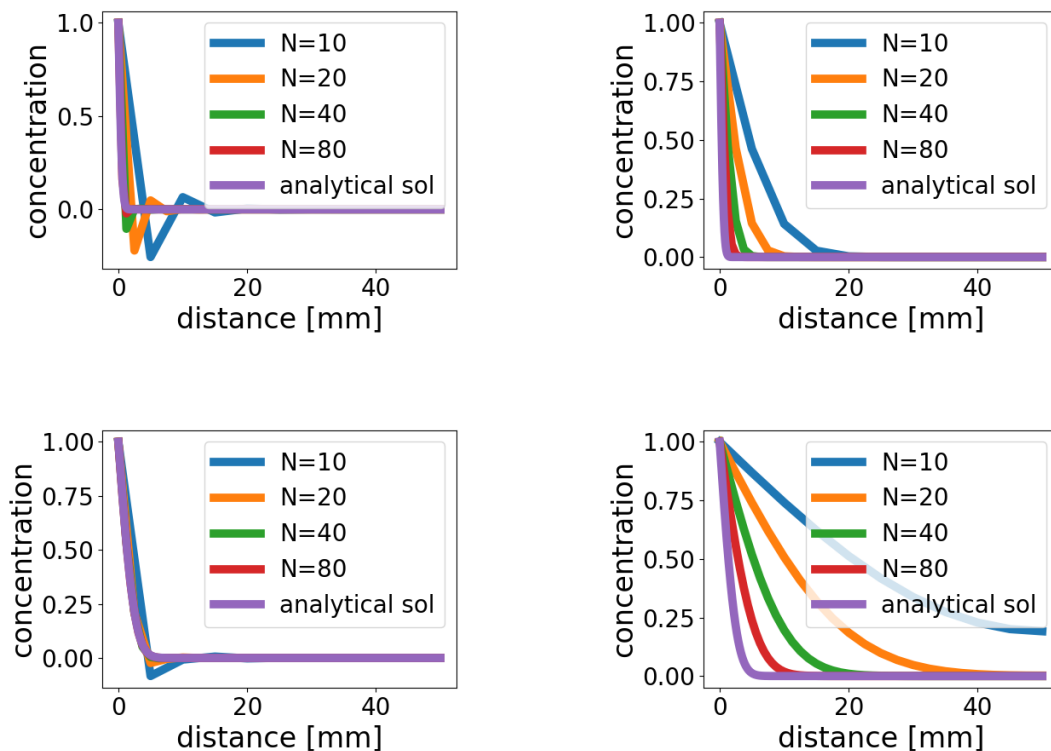
$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --
final_time 1800 --lumped lumped --check_point 0.1

$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --
final_time 32400 --lumped not --check_point 0.1

$ python diffusion_1D.py --L_max 50 --Ns 10 20 40 80 --dt 300 --
final_time 32400 --lumped lumped --check_point 0.1
```

Executes a series of numerical 1D diffusion simulations under varying conditions: two different simulation durations (short: 1800 seconds; long: 32400 seconds) and using both non-lumped and lumped numerical schemes. These commands produce simulation outputs (such as graphs or data files) that illustrate the behaviour of the diffusion process under different parameter settings, providing a benchmark for validating the more complex 3D simulations later in the workflow.

The plots below show the numerical 1D diffusion results (for both lumped and non-lumped schemes) compared to the analytical solution, at varying mesh resolutions ($N=10, 20, 40, 80$) and time scales (1800s vs. 32400s).



These comparisons help validate the consistency and accuracy of each method under different grid sizes and simulation durations, providing insight into potential trade-offs in resolution and runtime.

1. Check and Copy Required Surface Files from Chapter 4

```
if [ -f ../chp4/lh.pial.stl -a -f ../chp4/rh.pial.stl -a
-f ../chp4/lh.white.stl -a -f ../chp4/rh.white.stl -a -f
../chp4/lh.ventricles.stl ];
then
    echo "Required chp4 files found"

    # Making folder to store sufaces from chp4
    mkdir -p surfaces

    # Copying surfaces files from chp4
    cp ../chp4/lh.pial.stl ../chp4/rh.pial.stl
    ../chp4/lh.white.stl ../chp4/rh.white.stl
    ../chp4/lh.ventricles.stl surfaces
else
    echo "Required chp4 files not found"
    exit
fi
```

Checks that the necessary STL surface files (pial, white, and ventricular surfaces) from Chapter 4 exist. If found, creates a “surfaces” folder and copies these files there, ensuring they are available for the subsequent meshing steps.

2. Check for Required DTI Data from Chapter 5

```
if [ -f ../chp5/clean-tensor.mgz ];  
then  
    echo "Required chp5 files found"  
else  
    echo "Required chp5 files not found"  
    exit  
fi
```

Verifies the existence of the cleaned DTI data file (“clean-tensor.mgz”) from Chapter 5. The script halts if the file is missing, ensuring that all prerequisite data are in place before proceeding.

3. Create Mesh Refinements

```
python3 create_mesh_refinements.py
```

Runs a script that generates finite element meshes at multiple resolutions (16, 32, 64, and 128). These resolution-specific meshes (e.g. “brain_16.mesh”, “brain_32.mesh”, etc.) will be processed individually in subsequent steps.

4. Process Mesh at 16 Resolution

```
mkdir -p tmp  
python3 ../chp4/convert_to_dolfin_mesh.py --meshfile  
brain_16.mesh --hdf5file brain_16.h5  
# Mark subdomains (add parcellation tags)  
python3 ../chp4/add_parcellations.py --in_hdf5  
brain_16.h5 --in_parcel ../chp4/wmparc.mgz --out_hdf5  
brain_16_tags.h5 --add 17 1028 1035 3028 3035  
# Integrate DTI data with the mesh  
python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-  
tensor.mgz --mesh brain_16_tags.h5 --label 1 0.4 0.6 --  
out DTI_16.h5  
# Run diffusion simulation (with both lumped and non-  
lumped schemes)  
python3 chp6-diffusion-mritracer.py --mesh DTI_16.h5 --  
lumped lumped --annotation uniform16lumped
```

```
python3 chp6-diffusion-mritracer.py --mesh DTI_16.h5 --lumped not --annotation uniform16notlumped
```

Processes the 16-resolution mesh by converting it to Dolfin's HDF5 format ("brain_16.h5"), adding parcellation tags to produce "brain_16_tags.h5," and integrating cleaned DTI data to yield "DTI_16.h5." Finally, runs diffusion simulations using both lumped and non-lumped schemes with appropriate annotations.

5. Process Mesh at 32 Resolution

```
python3 ../chp4/convert_to_dolfin_mesh.py --meshfile brain_32.mesh --hdf5file brain_32.h5
python3 ../chp4/add_parcellations.py --in_hdf5 brain_32.h5 --in_parcel ../chp4/wmparc.mgz --out_hdf5 brain_32_tags.h5 --add 17 1028 1035 3028 3035
python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-tensor.mgz --mesh brain_32_tags.h5 --label 1 0.4 0.6 --out DTI_32.h5
python3 chp6-diffusion-mritracer.py --mesh DTI_32.h5 --lumped lumped --annotation uniform32lumped
python3 chp6-diffusion-mritracer.py --mesh DTI_32.h5 --lumped not --annotation uniform32notlumped
```

Repeats the conversion, tagging, and DTI integration for the 32-resolution mesh, producing "brain_32.h5", "brain_32_tags.h5", and "DTI_32.h5," and then running corresponding diffusion simulations with two simulation variants.

6. Process Mesh at 64 Resolution

```
python3 ../chp4/convert_to_dolfin_mesh.py --meshfile brain_64.mesh --hdf5file brain_64.h5
python3 ../chp4/add_parcellations.py --in_hdf5 brain_64.h5 --in_parcel ../chp4/wmparc.mgz --out_hdf5 brain_64_tags.h5 --add 17 1028 1035 3028 3035
python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-tensor.mgz --mesh brain_64_tags.h5 --label 1 0.4 0.6 --out DTI_64.h5
python3 chp6-diffusion-mritracer.py --mesh DTI_64.h5 --lumped lumped --annotation uniform64lumped
python3 chp6-diffusion-mritracer.py --mesh DTI_64.h5 --lumped not --annotation uniform64notlumped
```

Processes the 64-resolution mesh in the same manner to generate “brain_64.h5”, “brain_64_tags.h5”, and “DTI_64.h5,” followed by diffusion simulations for both lumped and non-lumped schemes.

7. Process Mesh at 128 Resolution

```
python3 ../chp4/convert_to_dolfin_mesh.py --meshfile
brain_128.mesh --hdf5file brain_128.h5
python3 ../chp4/add_parcellations.py --in_hdf5
brain_128.h5 --in_parcel ../chp4/wmparc.mgz --out_hdf5
brain_128_tags.h5 --add 17 1028 1035 3028 3035
python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-
tensor.mgz --mesh brain_128_tags.h5 --label 1 0.4 0.6 --
out DTI_128.h5
python3 chp6-diffusion-mritracer.py --mesh DTI_128.h5 --
lumped lumped --annotation uniform128lumped
python3 chp6-diffusion-mritracer.py --mesh DTI_128.h5 --
lumped not --annotation uniform128notlumped
```

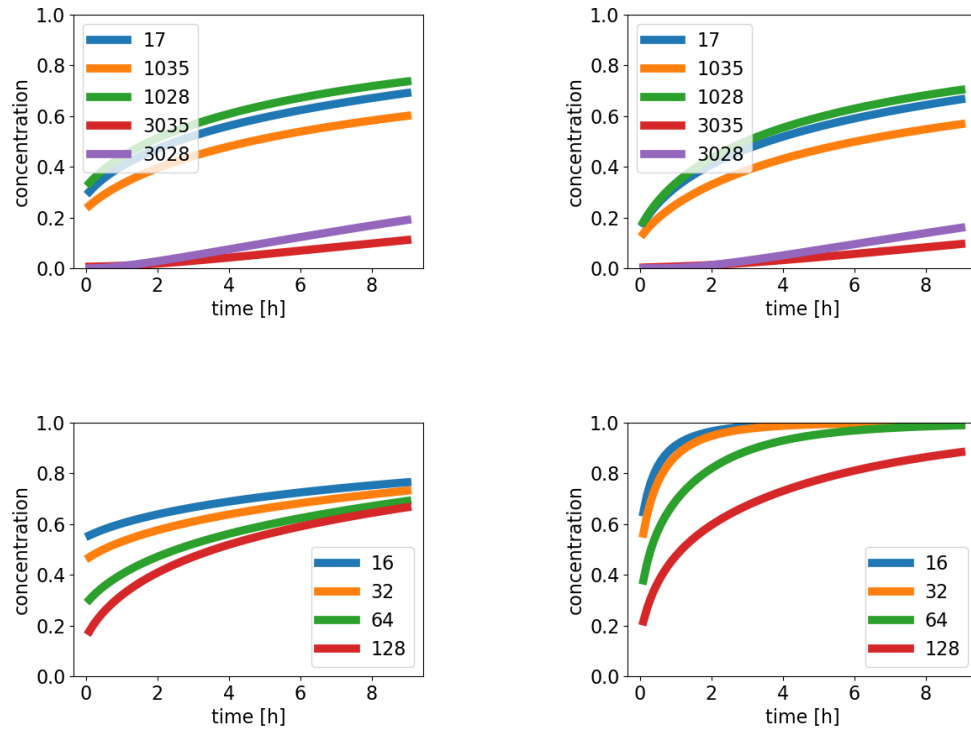
Processes the highest resolution (128) mesh to generate “brain_128.h5”, tag it as “brain_128_tags.h5”, integrate DTI data into “DTI_128.h5,” and execute diffusion simulations (both lumped and not). Note that the 128 mesh uses a significant amount of RAM.

8. Tracer Plotting for Uniform Meshes

```
python chp6-tracer-plot-regions.py
python chp6-tracer-plot-lump.py
```

Runs plotting scripts to generate visualisations of the tracer distribution and compare results between the lumped and non-lumped diffusion simulations for the uniformly refined meshes.

The plots below compare tracer concentration over time in two ways. First, for different anatomical tags (17, 1035, 1028, etc.) in uniformly refined meshes, highlighting the results of both lumped and non-lumped diffusion simulations. Second, for different mesh refinements (16, 32, 64, 128 elements in each dimension), illustrating how resolution influences the diffusion outcome.



We see distinct uptake curves for each tagged region under lumped vs. non-lumped approaches, while finer meshes yield more gradual curves, indicating improved accuracy and consistency compared to coarser discretisations.

9. Local Mesh Refinement Around the Hippocampus (Initial Copy and First Refinement)

```
cp brain_16.mesh Mesh_16.mesh
python3 ../chp4/convert_to_dolfin_mesh.py --meshfile
Mesh_16.mesh --hdf5file Mesh_16.h5
python3 ../chp4/add_parcellations.py --in_hdf5 Mesh_16.h5
--in_parcc ../chp4/wmparc.mgz --out_hdf5 Mesh_16_tags.h5 -
-add 17
python3 ../chp4/refine_mesh_tags.py --in_hdf5
Mesh_16_tags.h5 --out_hdf5 Mesh_16_ref1.h5 --refine_tag
17
python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-
tensor.mgz --mesh Mesh_16_ref1.h5 --label 1 0.4 0.6 --
out DTI_16_ref1.h5
```

Copies the 16-resolution mesh for local refinement around the hippocampus (tag 17), converts it to Dolfin format (“Mesh_16.h5”), adds parcellation information to produce “Mesh_16_tags.h5”, refines the mesh once (yielding “Mesh_16_ref1.h5”), and integrates DTI data to output “DTI_16_ref1.h5”.

10. Local Refinement: Second to Fourth Iterations

```
# Second refinement
python3 ../chp4/refine_mesh_tags.py --in_hdf5
Mesh_16_ref1.h5 --out_hdf5 Mesh_16_ref2.h5 --refine_tag
17

python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-
tensor.mgz --mesh Mesh_16_ref2.h5 --label 1 0.4 0.6 --
out DTI_16_ref2.h5

# Third refinement
python3 ../chp4/refine_mesh_tags.py --in_hdf5
Mesh_16_ref2.h5 --out_hdf5 Mesh_16_ref3.h5 --refine_tag
17

python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-
tensor.mgz --mesh Mesh_16_ref3.h5 --label 1 0.4 0.6 --
out DTI_16_ref3.h5

# Fourth refinement
python3 ../chp4/refine_mesh_tags.py --in_hdf5
Mesh_16_ref3.h5 --out_hdf5 Mesh_16_ref4.h5 --refine_tag
17

python3 ../chp5/dti_data_to_mesh.py --dti ../chp5/clean-
tensor.mgz --mesh Mesh_16_ref4.h5 --label 1 0.4 0.6 --
out DTI_16_ref4.h5
```

Performs additional iterations of local refinement focused on the hippocampus region. Each iteration produces a refined mesh (Mesh_16_ref2.h5, Mesh_16_ref3.h5, Mesh_16_ref4.h5) and corresponding DTI-integrated mesh files (DTI_16_ref2.h5, DTI_16_ref3.h5, DTI_16_ref4.h5) with progressively higher local resolution.

11. Run Diffusion Simulations on Locally Refined Meshes

```
# For lumped scheme
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref1.h5 --lumped lumped --
annotation=DTI16lumped_ref1

python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref2.h5 --lumped lumped --
annotation=DTI16lumped_ref2

python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref3.h5 --lumped lumped --
annotation=DTI16lumped_ref3
```

```
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref4.h5 --lumped lumped --
annotation=DTI16lumped_ref4

# For lumped scheme
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref1.h5 --lumped not --
annotation=DTI16notlumped_ref1
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref2.h5 --lumped not --
annotation=DTI16notlumped_ref2
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref3.h5 --lumped not --
annotation=DTI16notlumped_ref3
python3 chp6-diffusion-mritracer-ref.py --mesh
DTI_16_ref4.h5 --lumped not --
annotation=DTI16notlumped_ref4
```

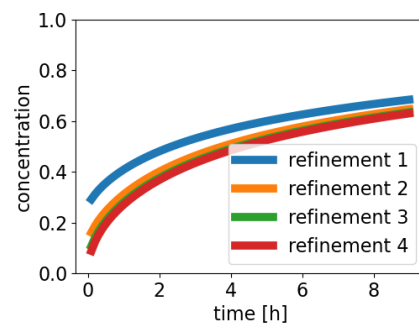
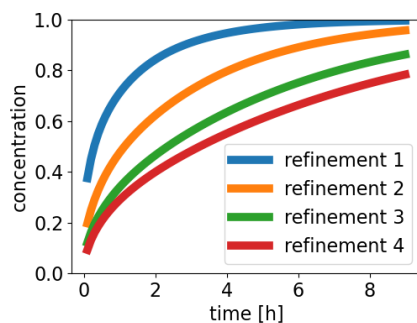
Runs diffusion simulations on each locally refined DTI-integrated mesh using both lumped and non-lumped numerical schemes. The outputs are annotated (e.g. “DTI16lumped_ref1”, “DTI16notlumped_ref1”, etc.) to distinguish results across successive refinement levels.

12. Plot Tracer Results for Locally Refined Meshes

```
python3 chp6-tracer-plot-ref.py
```

Executes a plotting script that processes simulation data from the locally refined meshes and generates visualisations summarising tracer distributions. These plots help assess the impact of local refinement on simulation outcomes.

The plots below compare tracer concentration over time for different levels of local mesh refinement (refinement 1–4), illustrating how increasing resolution influences the simulation results.



We observe that finer local refinements yield more convergent tracer profiles, indicating enhanced numerical accuracy in the regions of interest.

Each command group in Chapter 6, contained within the `all.sh` script, automates a comprehensive workflow that:

- Verifies and gathers required input files from Chapters 4 and 5.
- Generates meshes at multiple resolutions (both uniform and locally refined around the hippocampus).
- Converts meshes to simulation-ready formats, tags anatomical regions, and integrates DTI data.
- Runs diffusion simulations under varying numerical schemes.
- Produces plots to visualise and compare tracer distributions across different mesh refinements.

This integrated pipeline bridges segmentation, mesh generation, DTI processing, simulation, and visualisation, culminating in a detailed, simulation-ready brain model for studying anisotropic diffusion in heterogeneous brain regions.

7. Chapter 7

7.1. Overview

The final chapter, Chapter 7, broadens the perspective by exploring additional applications and future directions for the computational modeling pipeline. It discusses how the methods developed in the book can be extended to model other biophysical phenomena such as poroelasticity, electrophysiology, or traumatic brain injury. The chapter also addresses challenges related to multi-physics coupling—for instance, integrating cerebrospinal fluid (CSF) dynamics with brain tissue behaviour—and considers the potential for clinical translation in areas like patient-specific modeling and surgical planning. By synthesising the integration of imaging, mathematical modeling, and computational techniques, Chapter 7 highlights the transformative potential of these approaches to advance both our understanding of brain dynamics and the development of innovative therapeutic strategies.